6-1 · 실시간 데이터와 Kafka · 수업 전 10분

# 실습점검 · 환경재설정

## 지난 주 멈춤 · 9092 · 메모리 · 이미지

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

왜 점검하나

# 수업 전에 걸러 둘 것

① 지난 주 Spark · 레이크하우스 컨테이너가 떠 있으면 메모리를 나눠 쓴다

② 9092 를 다른 것이 잡고 있으면 오늘 브로커가 못 뜬다

③ apache/kafka:3.7.0 을 수업 중에 받으면 그만큼 기다린다

1~3단계는 확인 · 내려받기만 · 컨테이너는 실습 1단계에서 처음 만든다 · 재설정(5단계)은 처음부터 할 때만

흐름

# 다섯 단계 — 위에서부터 한 번씩

## 오늘은 컨테이너 하나 — 지난 주보다 가볍다

| 1. 지난 주 확인 | 2. 포트 · 메모리 | 3. 이미지 | 4. 오늘 만들 것 | 5. 환경 재설정 |
| --- | --- | --- | --- | --- |
| docker ps 제목 줄만 | 9092 비었나 | kafka 3.7.0 | 순서만 확인 | docker rm -fv |
| -a 는 Exited | 7.7GiB 안팎 | 210MB 받기 | 아직 치지 않음 | 처음부터 할 때만 |

1단계

# 지난 주 컨테이너가 멈춰 있는지 보세요

1. **docker ps --format 'table ...'**
   제목 줄(NAMES · STATUS · PORTS)만 — 떠 있는 것이 없다
2. **docker ps -a ... | grep -E 'NAMES|de-mysql|spark|minio|kafka'**
   5주차 넷 · 4주차 spark-iceberg · minio · de-mysql 이 Exited
3. **떠 있는 것이 있으면**
   그 폴더에서 docker compose stop — week5 · ~/lakehouse
4. **de-mysql 은**
   6-4 CDC 에서 다시 쓴다 — 멈춤만, 지우지 않는다

지난 주

**멈춤만**

**지우지 않음**

2단계 · 포트

# 9092 를 쓰는 다른 프로그램

01

## Windows (WSL2)

netstat -ano | findstr ":9092"

LISTENING 없으면 비어 있음

02

## macOS

lsof -nP -iTCP:9092

-sTCP:LISTEN

아무것도 없으면 비어 있음

03

## 리눅스

sudo ss -ltnp | grep ':9092 '

docker-proxy 면 컨테이너

**Windows 는 이 한 줄만 PowerShell 에서 · 나머지는 WSL 터미널에서**

2단계 · 메모리

# 8GB 로 잡았는데 7.749GiB 로 찍힌다

## docker info | grep -i 'total memory'

Docker 가 쓸 수 있는 메모리

| | |
|---|---|
| 실측 | Total Memory: 7.749GiB |
| 설정 | Docker Desktop 8GB |
| 판정 | 7.7GiB 안팎 이상이면 된다 |

VS

## 이번 주 늘어나는 것

오늘은 컨테이너 하나라 넉넉하다

| | |
|---|---|
| 6-1 · 6-2 | Kafka 하나 |
| 6-4 | + MySQL · Debezium |
| 6-5 | + MinIO |

3단계

# 이미지를 미리 받아 두세요

1. **docker pull apache/kafka:3.7.0**
   이미 받았으면 Image is up to date 로 끝난다

2. **docker images apache/kafka**
   DISK USAGE 612MB · CONTENT SIZE 210MB — 받는 양은 210MB

3. **6-4 를 위해 주말에**
   debezium/connect:2.7.3.Final — 2.2GB

받는 양

**210MB**

4단계 · 순서만

# 오늘 만드는 것 — 아직 치지 않는다

| 무엇 | 어디서 처음 만드나 | 남는가 |
|---|---|---|
| **~/data-eng/week6 폴더** | 첫 실행 1단계 | 예 — 이번 주 내내 |
| **kafka 컨테이너(9092)** | 첫 실행 1단계 | 아니오 — 정리에서<br>docker rm -fv kafka |
| **hello-stream 토픽<br>(파티션 3)과 메시지** | 첫 실행 2~5단계 | 아니오 — 컨테이너와<br>함께 사라짐 |

**컨테이너와 토픽은 「Kafka 핵심 구조와 첫 실행」 1 · 2단계에서 처음 만든다**

5단계

# 환경 재설정 — docker rm -fv kafka

## 컨테이너가 있을 때

kafka 한 줄이 찍히고 지워진다

- 토픽 · 메시지도 함께 사라진다
- 컨테이너 안 /tmp/kraft-combined-logs 에 있었다
- -v 가 이름 없는 볼륨 3개까지 지운다

VS

## 컨테이너가 없을 때

No such container: kafka

- 지울 것이 없다는 뜻 — 괜찮다
- docker ps -a --filter name=kafka 는 제목 줄만
- -f 만 쓰면 빈 볼륨이 쌓인다(실측)

6-1 · 수업 전 10분 마무리

# 다음은 실시간 데이터의 특징 (1H)

모아서 한꺼번에(배치)와 생기는 대로(스트리밍)는 무엇이 다른지부터 봅니다

- ✓ 배치 vs 스트리밍
- ✓ 이벤트 시간과 처리 시간
- ✓ Kafka 가 하는 일

막혔다면

**그 단계 아래 「막혔어요」 · 「질문」**